

# Documento de Calidad de Software según ISO/IEC 25010

---

## Sistema AFIP Mobile – Gestión Financiera Personal

**Versión:** 1.0.0

**Fecha:** Junio 2026

**Estándar:** ISO/IEC 25010:2011 – Systems and Software Quality Requirements and Evaluation (SQuaRE)

**Clasificación:** Documento Técnico de Calidad

---

## 1. Introducción

### 1.1 Propósito

Este documento presenta la evaluación de la calidad del producto software **AFIP Mobile** bajo el marco normativo de la ISO/IEC 25010, que define el modelo de calidad para sistemas y software. El objetivo es evidenciar el nivel de cumplimiento en cada característica y subcaracterística de calidad.

### 1.2 Descripción del Sistema

**AFIP Mobile** es una plataforma de gestión financiera personal que permite a los usuarios registrar ingresos, gastos, presupuestos, metas de ahorro y deudas. Está compuesto por:

- **API REST** (Node.js + Express 5 + Sequelize + MySQL)
- **Frontend Web** (React 18 + Vite + Tailwind CSS)
- **Aplicación Mobile** (Flutter 3.x / Dart)

### 1.3 Alcance de la Evaluación

La evaluación abarca el modelo de calidad del **producto software** (calidad interna, externa y en uso).

---

## 2. Modelo de Calidad ISO/IEC 25010

La norma define **8 características principales** de calidad, cada una con subcaracterísticas evaluadas:

```
ISO/IEC 25010
├─ 1. Adecuación Funcional
├─ 2. Eficiencia de Desempeño (Rendimiento)
├─ 3. Compatibilidad
├─ 4. Usabilidad
├─ 5. Confiabilidad
├─ 6. Seguridad
├─ 7. Mantenibilidad
└─ 8. Portabilidad
```

## 3. Escala de Evaluación

| Nivel         | Descripción                           | Representación |
|---------------|---------------------------------------|----------------|
| Excelente     | Supera los requisitos definidos       | ✓ ✓ ✓          |
| Satisfactorio | Cumple los requisitos definidos       | ✓ ✓            |
| Parcial       | Cumple parcialmente, requiere mejoras | ✓              |
| Insuficiente  | No cumple los requisitos              | ✗              |

## 4. Evaluación por Características

### 4.1 Adecuación Funcional

*Grado en que el software proporciona funciones que satisfacen las necesidades declaradas e implícitas del usuario.*

Subcaracterísticas

| Subcaracterística     | Descripción                                                                                               | Evidencia                                                                  | Nivel |
|-----------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|-------|
| Compleitud funcional  | El sistema cubre todos los módulos requeridos (ingresos, gastos, presupuestos, metas, deudas, categorías) | 9 módulos CRUD implementados completamente                                 | ✓✓✓   |
| Corrección funcional  | Los resultados de las operaciones son correctos                                                           | Lógica de negocio en controllers con validaciones; bcrypt para contraseñas | ✓✓    |
| Pertinencia funcional | Las funciones facilitan el objetivo del usuario                                                           | Módulos financieros directamente alineados con la gestión personal         | ✓✓✓   |

Evaluación General: ✓✓✓ Excelente

Métricas

- **Cobertura funcional:** 100% – Los 10 módulos de la BD tienen CRUD completo en la API.
- **Funciones sin errores:** ~95% (errores menores en validación de edge cases).

4.2 Eficiencia de Desempeño (Rendimiento)

Grado en que el rendimiento es adecuado en relación con los recursos utilizados.

Subcaracterísticas

| Subcaracterística       | Descripción                      | Evidencia                                                                      | Nivel |
|-------------------------|----------------------------------|--------------------------------------------------------------------------------|-------|
| Comportamiento temporal | Tiempo de respuesta y throughput | API responde en <200ms en condiciones locales; Express 5 con I/O no bloqueante | ✓✓    |
| Utilización de recursos | Consumo de CPU/RAM bajo carga    | Node.js event loop; Sequelize lazy loading                                     | ✓✓    |

|           |                                           |                                               |   |
|-----------|-------------------------------------------|-----------------------------------------------|---|
| Capacidad | Límites máximos de usuarios/transacciones | Sin límite implementado; depende del servidor | ✓ |
|-----------|-------------------------------------------|-----------------------------------------------|---|

Evaluación General: ✓ ✓ Satisfactorio

Métricas de Rendimiento

| Métrica                       | Valor Objetivo | Estado Actual                      | Observaciones            |
|-------------------------------|----------------|------------------------------------|--------------------------|
| Tiempo de respuesta API (p95) | < 500 ms       | ~150-300 ms (local)                | Depende de carga de red  |
| Tiempo de carga inicial (web) | < 3 s          | < 2 s (Vite con lazy loading)      | Bueno                    |
| Tiempo de carga app mobile    | < 5 s          | ~2-3 s (primera carga)             | Aceptable                |
| Consultas DB simultáneas      | > 50           | Limitado por MySQL connection pool | Mejorable con indexación |

Recomendaciones

- Implementar paginación en endpoints que devuelven listas grandes
- Agregar índices en columnas de búsqueda frecuente ( `user_FK` , `expense_date` )
- Considerar Redis para caché de datos frecuentes

4.3 Compatibilidad

Grado en que el software puede intercambiar información y/o realizar su función mientras comparte el mismo entorno.

Subcaracterísticas

| Subcaracterística | Descripción | Evidencia | Nivel |
|-------------------|-------------|-----------|-------|
|-------------------|-------------|-----------|-------|

|                          |                                                |                                                                                            |       |
|--------------------------|------------------------------------------------|--------------------------------------------------------------------------------------------|-------|
| <b>Coexistencia</b>      | Funciona sin conflictos junto a otros sistemas | API en puerto 3001, configurable vía <code>.env</code> ; no interfiere con otros servicios | ✓ ✓ ✓ |
| <b>Interoperabilidad</b> | Intercambio de datos con otros sistemas        | API REST + JSON estándar; CORS habilitado; compatible con cualquier cliente HTTP           | ✓ ✓ ✓ |

**Evaluación General:** ✓ ✓ ✓ **Excelente**

**Compatibilidad de Navegadores (Frontend Web)**

| Navegador       | Versión | Soporte    |
|-----------------|---------|------------|
| Google Chrome   | >= 90   | ✓ Completo |
| Mozilla Firefox | >= 88   | ✓ Completo |
| Microsoft Edge  | >= 90   | ✓ Completo |
| Safari          | >= 14   | ✓ Completo |

**Compatibilidad Mobile**

| Plataforma | Versión Mínima             | Soporte                                   |
|------------|----------------------------|-------------------------------------------|
| Android    | API Level 21 (Android 5.0) | ✓ Completo                                |
| iOS        | iOS 12.0                   | ✓ Completo (requiere macOS para compilar) |

**4.4 Usabilidad**

*Grado en que el software puede ser utilizado por usuarios específicos para lograr objetivos con efectividad, eficiencia y satisfacción.*

**Subcaracterísticas**

| Subcaracterística | Descripción | Evidencia | Nivel |
|-------------------|-------------|-----------|-------|
|-------------------|-------------|-----------|-------|

|                           |                                                 |                                                                              |     |
|---------------------------|-------------------------------------------------|------------------------------------------------------------------------------|-----|
| Reconocibilidad           | El usuario identifica si el sistema es adecuado | Interfaz clara con módulos identificados (Ingresos, Gastos, Presupuestos...) | ✓✓✓ |
| Aprendibilidad            | Facilidad de aprendizaje para nuevos usuarios   | Flujos simples de registro; formularios con validación inline                | ✓✓  |
| Operabilidad              | Facilidad de operación y control                | Navegación intuitiva; CRUD disponible en cada módulo                         | ✓✓  |
| Protección contra errores | El sistema previene errores del usuario         | Validación con <code>@hapi/joi</code> en backend; validación en formularios  | ✓✓  |
| Estética de la interfaz   | Presentación atractiva                          | Tailwind CSS; tipografía Google Fonts; diseño responsive en web y mobile     | ✓✓  |
| Accesibilidad             | Usabilidad por usuarios con discapacidades      | No evaluada formalmente; sin ARIA roles implementados                        | ✓   |

Evaluación General: ✓✓ Satisfactorio

Métricas de Usabilidad

| Métrica                               | Objetivo | Observaciones                         |
|---------------------------------------|----------|---------------------------------------|
| Tasa de completitud de tareas         | > 85%    | Flujos de registro intuitivos         |
| Errores por sesión (promedio)         | < 2      | Validaciones previenen mayor cantidad |
| Satisfacción del usuario (escala 1-5) | > 3.5    | Pendiente encuesta formal             |
| Tiempo en aprender función principal  | < 10 min | Módulos simples y bien nombrados      |

Recomendaciones

- Implementar atributos ARIA para accesibilidad
- Agregar mensajes de ayuda contextual (tooltips) en formularios complejos
- Implementar modo oscuro en la app mobile

## 4.5 Confiabilidad

Grado en que el sistema realiza funciones específicas bajo condiciones determinadas durante un período de tiempo.

### Subcaracterísticas

| Subcaracterística   | Descripción                                              | Evidencia                                                                     | Nivel |
|---------------------|----------------------------------------------------------|-------------------------------------------------------------------------------|-------|
| Madurez             | Frecuencia de fallos bajo operación normal               | Manejo de errores <code>try/catch</code> en todos los controllers             | ✓ ✓   |
| Disponibilidad      | Proporción de tiempo que el sistema está operativo       | PM2 garantiza reinicio automático ante fallos                                 | ✓ ✓   |
| Tolerancia a fallos | Sistema mantiene nivel de servicio ante fallos parciales | Errores de inicialización de categorías no interrumpen el registro de usuario | ✓ ✓   |
| Recuperabilidad     | Capacidad de recuperar datos en caso de fallo            | Procedimientos de backup y restore documentados                               | ✓ ✓   |

Evaluación General: ✓ ✓ Satisfactorio

### Métricas de Confiabilidad

| Métrica                           | Objetivo           | Observaciones                    |
|-----------------------------------|--------------------|----------------------------------|
| Uptime del sistema                | > 99% (producción) | Depende de PM2 + infraestructura |
| MTBF (Mean Time Between Failures) | > 720 horas        | No medido formalmente            |
| MTTR (Mean Time To Repair)        | < 4 horas          | Documentado en plan DRP          |

## 4.6 Seguridad

Grado en que el software protege la información y los datos para que las personas o sistemas tengan el nivel de acceso apropiado.

### Subcaracterísticas

| Subcaracterística     | Descripción                                  | Evidencia                                                                        | Nivel |
|-----------------------|----------------------------------------------|----------------------------------------------------------------------------------|-------|
| Confidencialidad      | Solo usuarios autorizados acceden a datos    | JWT en todas las rutas protegidas; contraseñas con bcryptjs (salt=10)            | ✓ ✓ ✓ |
| Integridad            | Protección contra modificación no autorizada | Validaciones con Joi; restricciones FK en MySQL; tokens con expiración 1h        | ✓ ✓   |
| No repudio            | Registro de acciones de usuarios             | morgan logea todas las peticiones HTTP                                           | ✓ ✓   |
| Autenticidad          | Verificación de identidad                    | JWT firmado con <code>JWT_SECRET</code> ; bcrypt para verificación de contraseña | ✓ ✓ ✓ |
| Rendición de cuentas  | Trazabilidad de acciones                     | <code>createdAt</code> / <code>updatedAt</code> en todos los registros de BD     | ✓ ✓   |
| Resistencia a ataques | Protección contra vulnerabilidades comunes   | Sanitización básica con Joi; CORS habilitado; sin protección CSRF explícita      | ✓     |

Evaluación General: ✓ ✓ Satisfactorio

### Métricas de Seguridad

| Aspecto                  | Implementación                                         | Nivel      |
|--------------------------|--------------------------------------------------------|------------|
| Hashing de contraseñas   | bcryptjs con salt=10 rounds                            | Alto       |
| Autenticación            | JWT con expiración 1 hora                              | Alto       |
| Autorización por usuario | Cada operación verifica <code>user_id</code> del token | Alto       |
| Transporte de datos      | HTTPS en producción (Let's Encrypt)                    | Alto       |
| Inyección SQL            | ORM Sequelize con queries parametrizadas               | Alto       |
| Rate limiting            | No implementado                                        | Deficiente |

|                 |                 |            |
|-----------------|-----------------|------------|
| CSRF protection | No implementado | Deficiente |
|-----------------|-----------------|------------|

Recomendaciones

- Implementar rate limiting (ej. `express-rate-limit` )
- Agregar protección CSRF para el frontend web
- Implementar lista de tokens revocados (blacklist JWT) o refresh tokens
- Añadir validación de fuerza de contraseña

4.7 Mantenibilidad

Grado de efectividad y eficiencia con la que el software puede ser modificado.

Subcaracterísticas

| Subcaracterística    | Descripción                                      | Evidencia                                                                           | Nivel |
|----------------------|--------------------------------------------------|-------------------------------------------------------------------------------------|-------|
| Modularidad          | Componentes independientes con bajo acoplamiento | Estructura MVC clara: controllers/, models/, routers/, schemes/                     | ✓✓✓   |
| Reusabilidad         | Componentes reutilizables                        | Middlewares genéricos (auth.middleware, user.middleware); servicios React separados | ✓✓✓   |
| Analizabilidad       | Facilidad para diagnosticar deficiencias         | Estructura de carpetas predecible; morgan para logs de peticiones                   | ✓✓    |
| Modificabilidad      | Facilidad para implementar cambios               | Patrón MVC facilita cambios sin impacto en capas adyacentes                         | ✓✓✓   |
| Capacidad de pruebas | Facilidad para verificar mediante pruebas        | Arquitectura separada facilita unit testing; sin tests unitarios implementados aún  | ✓     |

Evaluación General: ✓✓✓ Excelente

### Métricas de Mantenibilidad

| Métrica                          | Valor                 | Observaciones                               |
|----------------------------------|-----------------------|---------------------------------------------|
| Complejidad ciclomática promedio | Baja-Media            | Controllers con funciones atómicas          |
| Líneas por función (promedio)    | <50 líneas            | Funciones concisas                          |
| Dependencias externas            | 8 packages producción | Gestionadas con npm                         |
| Deuda técnica estimada           | Baja-Media            | Modularidad buena; falta cobertura de tests |

### 4.8 Portabilidad

Grado en que el software puede ser transferido de un entorno a otro.

### Subcaracterísticas

| Subcaracterística       | Descripción                                     | Evidencia                                                           | Nivel |
|-------------------------|-------------------------------------------------|---------------------------------------------------------------------|-------|
| <b>Adaptabilidad</b>    | Se adapta a diferentes entornos                 | Variables de entorno configurables (.env); soporte multi-plataforma | ✓ ✓ ✓ |
| <b>Instalabilidad</b>   | Facilidad de instalación en entornos definidos  | npm install; flutter pub get; scripts SQL                           | ✓ ✓ ✓ |
| <b>Reemplazabilidad</b> | Puede reemplazar otro software del mismo ámbito | API REST estándar; frontend React intercambiable                    | ✓ ✓   |

**Evaluación General:** ✓ ✓ ✓ Excelente

### Entornos Soportados

| Componente   | Windows     | Linux     | macOS              |
|--------------|-------------|-----------|--------------------|
| API Node.js  | ✓           | ✓         | ✓                  |
| MySQL        | ✓           | ✓         | ✓                  |
| Frontend Web | ✓ (build)   | ✓ (build) | ✓ (build)          |
| App Flutter  | ✓ (Windows) | ✓ (Linux) | ✓ (macOS)          |
| App iOS      | ✗           | ✗         | ✓ (requiere Xcode) |

## 5. Resumen de Evaluación

| N° | Característica ISO 25010 | Nivel de Cumplimiento | Puntuación (1-5) |
|----|--------------------------|-----------------------|------------------|
| 1  | Adecuación Funcional     | ✓ ✓ ✓ Excelente       | 5/5              |
| 2  | Eficiencia de Desempeño  | ✓ ✓ Satisfactorio     | 3.5/5            |
| 3  | Compatibilidad           | ✓ ✓ ✓ Excelente       | 5/5              |
| 4  | Usabilidad               | ✓ ✓ Satisfactorio     | 3.5/5            |
| 5  | Confiabilidad            | ✓ ✓ Satisfactorio     | 3.5/5            |
| 6  | Seguridad                | ✓ ✓ Satisfactorio     | 3.5/5            |
| 7  | Mantenibilidad           | ✓ ✓ ✓ Excelente       | 5/5              |
| 8  | Portabilidad             | ✓ ✓ ✓ Excelente       | 5/5              |

**Puntuación Global: 4.19 / 5.00 = 83.8% de cumplimiento**

## 6. Plan de Mejora de Calidad

| Prioridad | Área | Mejora Propuesta | Impacto | Esfuerzo |
|-----------|------|------------------|---------|----------|
|-----------|------|------------------|---------|----------|

|       |               |                                      |       |       |
|-------|---------------|--------------------------------------|-------|-------|
| Alta  | Seguridad     | Implementar rate limiting            | Alto  | Bajo  |
| Alta  | Seguridad     | Refresh tokens / revocación JWT      | Alto  | Medio |
| Media | Accesibilidad | Atributos ARIA en frontend web       | Medio | Bajo  |
| Media | Rendimiento   | Paginación en listas grandes         | Medio | Medio |
| Media | Confiabilidad | Implementar suite de tests unitarios | Alto  | Alto  |
| Baja  | Rendimiento   | Caché con Redis                      | Medio | Alto  |
| Baja  | Seguridad     | Protección CSRF                      | Medio | Medio |

## 7. Conclusiones

El sistema **AFIP Mobile** demuestra un nivel de calidad **satisfactorio a excelente** en la mayoría de las características evaluadas según ISO/IEC 25010. Sus principales fortalezas son:

- **Arquitectura modular y mantenible** con separación clara entre capas
- **Seguridad funcional** con JWT y bcrypt correctamente implementados
- **Alta portabilidad** gracias al uso de tecnologías multiplataforma
- **Funcionalidad completa** que cubre todos los módulos del modelo de dominio

Las áreas de mejora prioritarias se centran en **seguridad avanzada** (rate limiting, CSRF) y **calidad de testing** (implementación de pruebas unitarias e integración).

*Documento elaborado conforme a la norma ISO/IEC 25010:2011 – AFIP Mobile v1.0.0 – Junio 2026*